

React Native Lead — Interview Revision Guide

Targeted Crux Notes & Deep-Dive Technical Blueprint | Kirana Club Context

Target Application Profile (Kirana Club Lens):

- **Users:** Tier 2/3/4 Indian kirana owners on low-end Android devices (■8,000–13,000 range, ~2–3 GB RAM).
- **Network:** Highly constrained, high-latency, unreliable cellular networks (2G/3G/spotty 4G).
- **Content:** Media-heavy feed (Images, Videos, Community Posts, B2B Commerce catalog, 1Cr+ downloads).
- **Core Challenge:** Delivering 60fps fast-loading feeds on weak CPUs/RAM without memory leaks or scroll jank.

1. React Native Internals & Architecture Evolution

Why UI Lag Happens

- **Single-Threaded JS Model:** The JS thread handles business logic, state updates, API parsing, and component rendering calculations. If blocked, frame generation stops, causing UI jank.
- **Major Blockers:** Synchronous heavy CPU computation, parsing large deeply-nested JSON payloads, promise queue starvation, and JS-driven layout/animation calculations.

JS Event Loop: Micro-tasks vs. Macro-tasks

- **Micro-tasks:** Promise callbacks, queueMicrotask. Processed immediately after current execution stack empties, before the next macro-task.
- **Macro-tasks:** setTimeout, setInterval, I/O events. Executed in subsequent event loop ticks.
- *Interview Insight:* Uncontrolled Promise chains clog the micro-task queue, completely starving render updates.

Old Architecture (Bridge) vs. New Architecture

Feature	Old Architecture (Bridge)	New Architecture (JSI / Fabric / Turbo)
Communication	Async JSON serialization over bridge queue.	Direct synchronous C++ calls via JSI .
Native Modules	Eager initialization on app launch.	TurboModules: Lazy loaded on-demand.
UI Renderer	Paper ViewManager (Async over bridge).	Fabric: Concurrent, thread-safe, sync layout.
JS Engine	JSC / Hermes standard mode.	Hermes pre-compiled bytecode (Fast TTI, low RAM).

Core Architecture Deep-Dive Terminology

- **JSI (JavaScript Interface):** Replaces the asynchronous JSON bridge with C++ host objects. Enables JS to hold direct memory references to native methods for synchronous execution.
- **Fabric:** Direct native UI renderer. Allows React to measure and manipulate native view trees synchronously without bridge serialization.
- **Hermes:** A light JS engine tuned for React Native. Pre-compiles JS into bytecode at build time, eliminating runtime parsing, lowering RAM usage, and accelerating Time-To-Interactive (TTI).
- **Yoga:** Cross-platform layout engine (written in C++) that converts CSS Flexbox declarations into absolute native view coordinates.

Network Path of an API Call

- **Flow:** fetch/Axios (JS) → JS Wrapper → **JSI/Bridge** → Native Networking (OkHttp / NSURLSession) → Network → Response parsing → **JSI/Bridge** → JS Thread (Promise.resolve).
- **Cost Drivers:** Stringifying/parsing large JSON strings across the JS boundary and JS Heap object allocation overhead during Garbage Collection.

2. Performance Tuning & Rendering Optimization

- **Key Metrics:** Measure FPS (JS vs UI thread), Cold Start TTI, Scroll Jank, and Memory Heap spikes (GC pauses).
- **Re-Render Control:** Leverage React.memo with custom comparison functions, useCallback and useMemo for referential stability, and component granularity to isolate state updates.
- **Memory Leaks & GC:** Uncleared timers (setInterval), orphaned DeviceEventEmitter listeners, and un-purged JS-held image caches cause severe memory leaks and GC frame stutter on 2GB RAM phones.

3. List Rendering: FlatList vs. FlashList

- **FlatList Tuning:** Use getItemLayout for fixed height items (bypasses layout measurement), reduce windowSize (default 21 → 5-7 on low-end Android), tune maxToRenderPerBatch, and set removeClippedSubviews={true}.
- **FlashList (Shopify):** Employs **Cell Recycling** (reuses existing native views like Android RecyclerView) instead of unmounting/remounting elements. Drastically cuts native view creation cost and prevents blank spaces during fast scrolling.

4. Media Optimization (Images & Video) on Weak Networks

- **Images - Sizing & Caching:** Request exact-sized image variants from CDN based on screen density using PixelRatio.getPixelSizeForLayoutSize(). Rely on FastImage (Glide/SDWebImage) for aggressive disk/RAM caching.
- **Images - Placeholders & Rendering:** Use **BlurHash** or 50-byte low-res base64 thumbnails (LQIP) while streaming full images. Avoid borderRadius on massive images (causes expensive off-screen rendering on Android); instead use overlay shapes.
- **Video - Adaptive Streaming:** Use Adaptive Bitrate Streaming (HLS/DASH) to automatically downgrade resolution (e.g., to 360p) on 2G/3G connections.
- **Video - Memory Management:** Low-end devices (2-3GB RAM) can only decode 1-2 videos simultaneously. Enforce strict pre-buffer limits (max 1 upcoming video) and use exact viewport visibility to auto-pause off-screen players, preventing OOM crashes.

5. App Boot Time (Cold Start) Optimization

- **Hermes Engine:** Ensure Hermes is enabled to pre-compile JS into bytecode at build time. This bypasses the heavy JS parsing step on device, reducing TTI (Time-To-Interactive) significantly.
- **Lazy Loading & TurboModules:** Migrate from eager-loaded native modules to JSI-based **TurboModules** (lazy instantiation). Lazy load JS components via React.lazy that are not immediately visible on the initial screen.

- **Asset & API Strategy:** Inline small critical icons as SVGs. Delay loading heavy custom fonts until layout completes. Fetch initial feed data synchronously from **MMKV** local storage on boot to display UI instantly, then revalidate via network in the background.

6. Animations & Threading

- **useNativeDriver: true:** Offloads frame updates directly to the UI thread (Android Choreographer). Keeps animations smooth even if the JS thread is frozen. Limited to non-layout properties (transform, opacity).
- **Reanimated (v2/v3):** Executes JS **Worklets** on a dedicated UI JS runtime context using JSI, allowing high-performance gesture animations without main JS thread bottleneck.

7. State Management, Storage & Persistence

Storage Metric	AsyncStorage	MMKV (Tencent / react-native-mmkv)
Architecture	Async calls via SQLite/SharedPreferences over bridge.	Synchronous C++ memory-mapped file access via JSI.
Read/Write Speed	Slow (Async overhead, string serialization).	~10x–30x faster (Direct memory access).
Thread Model	Asynchronous (Promises).	Synchronous direct read/write.

- **Offline Architecture:** Serve cached feed data instantly from MMKV on cold start, perform optimistic UI updates, and queue background requests with write-ahead logs.

8. Hands-on System Design: Scroll Event Telemetry & Batching

Scenario: Track scroll viewport viewability for items without analytics SDK, resilient to crashes and offline state.

```
import { MMKV } from 'react-native-mmkv';
const storage = new MMKV();
let eventQueue: Event[] = [];
const BATCH_SIZE = 20;

export const trackView = (itemId: string) => {
  eventQueue.push({ itemId, timestamp: Date.now() });
  if (eventQueue.length >= BATCH_SIZE) flushEvents();
};

const flushEvents = async () => {
  if (eventQueue.length === 0) return;
  const payload = [...eventQueue];
  eventQueue = [];
  storage.set('pending_analytics', JSON.stringify(payload)); // MMKV Fallback
  try {
    await api.post('/analytics/batch', { events: payload });
    storage.delete('pending_analytics');
  } catch (err) { /* Retry on next tick/launch */ }
};
```

9. Shipping, SDUI & WebView Performance

- **Server-Driven UI (SDUI):** Backend sends JSON layout contracts. Protect older apps with strict client side schema validation (Zod) and component fallbacks to prevent crashes.
- **OTA Updates (CodePush / Expo Updates):** Instantly update JS bundle logic and assets. *Hard Limit:* Native code changes (Java/Kotlin/ObjC), new native packages, or native permissions require full store release.
- **WebView Trade-offs on Low-End Devices:**
 - **High Memory Footprint:** WebViews launch a separate Chromium instance on Android, consuming ~100MB+ RAM just to boot, risking OOM crashes on 2GB RAM phones.
 - **Cold Start Penalty:** Booting the WebView engine adds 500ms–1s of pure overhead.
 - **Lost Native Responsiveness:** Multi-touch, list virtualization (FlashList), and hardware-accelerated animations are absent.
 - **Strategy:** Use strictly for static, low-frequency content (Terms & Conditions, Help). Use Native/SDUI for interactive core feeds.

10. Observability & AI Workflow in RN Lead Role

- **Observability:** Instrument Sentry/Crashlytics for dual JS + Native stack traces. Profile rendering via Flipper and Android Studio Profiler.
- **AI Workflow:** Leverage AI for boilerplate creation, unit testing, and contract verification while maintaining strict human code review for performance edge cases.